

Frontend Internship Assignment

Build a small React app that takes a free-form text input, sends it to an AI model, and turns the result into an interactive tool.

Calling the model is the easy part. We're looking at how you turn unpredictable AI output into reliable UI, and how you handle it when the model gets things wrong.

Time: aim for ~8 hours total. Please don't spend more than 8 hours of actual work. If you run out, stop and note what you'd do next.

What to build

Pick one — same requirements for each, so choose whichever you'd enjoy.

- **Study assistant** — user pastes notes or a topic; the AI generates flashcards or a quiz; your app lets them flip through cards, take the quiz, and re-test wrong answers.
- **Fridge-to-recipe** — user lists ingredients; the AI returns a recipe; your app shows steps to check off, scalable servings, and ingredient swaps.
- **Trip planner** — user describes a trip; the AI returns a day-by-day itinerary; your app lets them expand, remove, and reorder stops.

How you approach it is up to you — your own UI, structure, and methods are all welcome. The one firm rule: it can't be a chatbot. The AI should return **structured data** (e.g. JSON) that your code parses and renders as interactive components; printing the model's raw text in a chat box doesn't meet the requirement.

Requirements

- React (hooks, functional components).
- A free-form text input.
- A real LLM API (any provider — see below).
- The model returns structured data; your app parses it and renders interactive, stateful UI.

- Handles the model returning bad output — malformed JSON, wrong shape, empty, slow, or failed. No crashes; show an error or a retry; don't let a stale response overwrite a newer one.
- Loading, error, and empty states.
- Works on mobile.
- A README with setup, an AI-usage note, known limitations, and time spent.

Most of the signal is in that fourth-from-last point. Handling failure well is what separates people who've built AI features from those who haven't.

Stretch (optional)

- Let the AI return different kinds of blocks (a card, a chart, a checklist) and render each appropriately.
- Stream the result as it generates.
- A refinement loop: follow-up prompts that edit the existing result instead of regenerating.
- Save and reload sessions.
- Polish: animation, dark mode, keyboard navigation.

Stack

- **Frontend:** React with hooks. TypeScript is optional and not graded.
- **AI:** any provider. Gemini, Groq, and OpenRouter have free tiers; OpenAI and Anthropic are cheap. Or run a model locally for free with [Ollama](#) — note that smaller local models are less consistent at structured output, so your failure handling matters more.
- **Tooling:** anything that helps — Vite/Next, a CSS framework, AI SDKs. If an SDK handles structured output or streaming for you, be ready to explain what it's doing.
- **API key:** don't ship it in the browser. Route the model call through a small backend or serverless function. We'll look at this.

AI tools

Use them — Copilot, Cursor, Claude, ChatGPT, whatever you normally use. In the interview we'll ask you to explain decisions, fix a bug, and add a small feature, so don't ship code you don't understand.

Add a short note in your README on what you used AI for — being honest about it counts in your favor.

Original work

The work should be your own. Referencing docs, tutorials, and AI assistants is fine, but submitting an existing project — a public repo, a tutorial result, or someone else's code — as your own is not. Since you'll explain and extend your code live in the interview, anything you didn't build yourself will be clear.

Submitting

- A public GitHub repo (or private with access). Small, meaningful commits beat one large one.
- A README (setup, usage, AI-usage note, limitations, time spent).
- A short screen recording showing the app working, and instructions to run it locally (`npm install && npm start` should work).

How we evaluate

Area	Weight
React & frontend architecture	25%
AI integration & data handling	25%
Handling bad AI output	20%
UI/UX & product sense	15%
Communication & understanding	15%

A clean, solid core beats a pile of half-working features.

Interview

If your submission moves forward, expect to demo it, walk through your code, review a short AI-generated snippet, fix a bug we introduce, and add a small feature.

FAQ

TypeScript required? No.

AI tools allowed? Yes — just understand your code.

AI SDK allowed? Yes — be ready to explain what it does.

Which model? Any. It doesn't affect your score.

Authentication? Not needed.

Deploy required? Preferred but optional.

No money for credits? Use a free tier or run Ollama locally.

Questions are welcome.